

软件过程与管理课程作业

作业编号: Assgt_01

姓名: 刘林宜 学号: 2023141461115

一、项目背景与概述

本次作业选取的项目为: ORB-SLAM3 单目 SLAM 系统在 Windows 平台下的部署与实验。ORB-SLAM3 是一种基于特征点的视觉 SLAM (Simultaneous Localization and Mapping) 系统, 广泛应用于机器人定位与三维重建等领域。本项目的主要任务是在 Windows 操作系统环境下完成该系统的部署, 并利用公开数据集进行单目 SLAM 实验验证。

1.1 项目目标

- ◆ 完成 ORB-SLAM3 在 Windows 平台下的编译与运行
- ◆ 配置并整合相关依赖库 (如 OpenCV、Eigen、Pangolin 等)
- ◆ 使用公开数据集进行单目 SLAM 实验
- ◆ 对实验结果进行分析与验证

1.2 项目特点

- ◆ 项目规模较小, 属于课程实验类项目
- ◆ 开发周期较短 (数周内完成)
- ◆ 以实验验证为主要目标, 而非工程化产品开发

二、软件过程定义

2.1 需求分析过程

需求分析过程是软件开发的起点, 其主要目标是明确系统需要完成的功能及约束条件。

主要活动:

- ◆ 明确项目总体目标 (成功运行 ORB-SLAM3 系统)
- ◆ 确定实验类型 (单目 SLAM)
- ◆ 确定运行平台 (Windows 操作系统)

- ◆ 分析系统运行所需的依赖环境（OpenCV、Eigen、Pangolin 等）
- ◆ 确定输入数据（公开数据集）及输出结果形式（轨迹、地图）

输出成果：

- ◆ 非正式需求说明
- ◆ 实验目标描述

特点：

由于项目规模较小，该阶段未形成正式的需求规格说明书，而是以笔记和任务目标的形式存在。

2.2 系统设计过程

系统设计过程主要用于确定系统的结构及实现方案。

主要活动：

- ◆ 分析 ORB-SLAM3 系统整体架构（Tracking、Local Mapping、Loop Closing）
- ◆ 研究系统模块之间的关系与数据流
- ◆ 设计 Windows 平台下的编译与构建方案（CMake + Visual Studio）
- ◆ 规划数据输入输出流程（数据集读取、轨迹输出）

输出成果：

- ◆ 系统结构理解模型
- ◆ 编译与运行方案设计

特点：

该阶段以“理解现有系统 + 平台适配”为主，而非从零设计系统架构。

2.3 实现过程

实现过程是软件开发的阶段，主要完成代码编译、环境配置及功能实现。

主要活动：

- ◆ 搭建开发环境（Visual Studio、CMake）
- ◆ 安装并配置依赖库（OpenCV、Eigen 等）
- ◆ 编译 ORB-SLAM3 源代码
- ◆ 修改部分代码以适配 Windows 平台
- ◆ 构建可执行程序

输出成果：

- ◆ 可成功运行的 ORB-SLAM3 系统

特点：

实现过程不仅包括代码开发，还包含大量环境配置与调试工作。

2.4 测试过程

测试过程用于验证系统功能是否正确实现。

主要活动：

- ◆ 使用公开数据集（如 TUM 数据集）进行实验
- ◆ 检查系统是否能够正常运行
- ◆ 验证轨迹输出结果的合理性
- ◆ 发现并修复运行过程中出现的问题

输出成果：

- ◆ 实验运行结果
- ◆ 问题记录与修复情况

2.5 验证与分析过程

验证过程用于判断系统是否满足预期目标。

主要活动：

- ◆ 对 SLAM 输出轨迹进行分析
- ◆ 对比实验结果与预期效果
- ◆ 分析误差来源（如特征匹配误差、数据质量等）

输出成果：

- ◆ 实验分析报告
- ◆ 结果评价结论

2.6 项目管理过程

项目管理过程用于保障项目按计划进行。

主要活动：

- ◆ 制定简单时间计划

- ◆ 划分任务阶段（环境搭建、调试、实验）
- ◆ 跟踪项目进度

2.7 支持过程

（1）配置管理支持

- ◆ 使用 Git 进行基础版本管理
- ◆ 对关键修改进行简单记录

（2）问题解决过程

- ◆ 在编译与运行过程中不断发现问题
- ◆ 通过查阅资料与调试进行解决

（3）文档支持

- ◆ 记录关键配置步骤
- ◆ 保存实验结果

项目的软件过程整体呈现为：

以生命周期模型为框架 + 以迭代调试为驱动的轻量级过程模型

三、软件过程定义的来源

3.1 标准参考

本项目的软件过程定义主要参考了 ISO/IEC 12207 标准，其具体体现在多个方面：首先，在生命周期阶段划分上，按照标准将软件开发过程划分为需求分析、系统设计、实现以及测试等主要阶段；其次，在过程组织上，采用结构化方式对各类活动进行系统划分与安排；此外，还引入了支持过程与管理过程的基本概念，使软件过程不仅涵盖开发活动本身，还包括必要的支撑与管理内容，从而提升过程的完整性与规范性。

3.2 课程讲授内容

通过课程学习，系统掌握了软件过程的层次结构、生命周期模型以及软件过程的分类方法等核心内容。其中，软件过程的层次结构有助于从整体到局部对开发活动进行分解与组织，生命周期模型为项目各阶段的划分提供了基本框架，而软件过程分类方法则帮助明确不同类型过程的职责与作用。这些理论知识为本项

目的软件过程划分、结构设计以及各阶段活动的组织提供了重要的指导，使得所定义的软件过程具有一定的规范性与合理性。

四、软件过程的剪裁

4.1 剪裁原则

- ◆ 保留核心开发过程
- ◆ 简化支持过程与管理过程
- ◆ 删除不必要的复杂流程

4.2 剪裁内容分析

(1) 文档过程的简化

在标准软件过程中，通常需要编写完整的需求规格说明书、设计文档以及测试文档等，但在本项目中并未形成正式的文档体系，仅以简单记录、笔记等形式保留关键信息。这种简化主要是由于项目周期较短、任务目标明确，同时完整文档编写所需成本较高，在当前项目规模下收益有限，因此采用了轻量化的文档方式。

(2) 配置管理过程的弱化

标准过程通常包括配置项识别、版本控制、变更控制以及配置审计等内容，而本项目中仅使用 Git 进行基础的版本管理，对代码修改进行简单记录，未开展正式的配置审计与变更控制流程。这主要是因为项目规模较小、开发人员较少，系统复杂度和变更频率较低，采用简化的配置管理方式已能够满足实际需求。

(3) 验证与确认过程的合并

在标准软件工程中，验证与确认通常作为两个独立过程存在，用于分别检查“是否正确实现”和“是否满足需求”。而在本项目中，这两类活动通过实验运行统一完成，即通过对系统运行结果的观察与分析，同时验证功能实现的正确性以及是否达到预期目标。这种合并主要源于项目需求较为简单且缺乏明确的外部用户场景。

(4) 维护过程的省略

标准生命周期中通常包含软件维护过程，用于支持系统上线后的持续更新与改进。但本项目属于课程实验性质，在完成既定目标后即结束，不涉及后续的系统维护与版本迭代。因此在过程定义中省略了维护阶段，这与项目的一次性特点相一致。

(5) 管理过程的简化

在标准软件过程中，项目管理通常包括风险管理、质量保证、评审机制等多个方面。而在本项目中，仅进行了简单的进度安排与任务划分，未建立正式的风险管理与质量保证流程。这主要是由于项目参与人员较少、组织结构简单，同时项目周期较短，为降低管理成本并提高开发效率，对管理过程进行了适当简化。

4.3 剪裁前后对比分析

过程类别	标准过程	剪裁后
需求过程	完整需求文档	简化为目标描述
设计过程	详细设计	理解现有系统
实现过程	完整开发	以调试为主
测试过程	系统测试	实验验证
配置管理	完整流程	基础 Git
维护过程	持续维护	不涉及

五、过程合理性分析

5.1 与项目规模匹配

本项目属于课程实验性质的小规模软件项目，开发周期较短、参与人员较少，系统复杂度相对有限。因此，在软件过程选择上采用了轻量级过程模型，对标准过程进行了适当剪裁，以避免引入过度复杂的工程化流程。通过这种方式，使软件过程能够更贴合实际开发条件，在保证基本开发活动完整性的同时，提高执行效率与灵活性。

5.2 与项目目标匹配

本项目的核心目标是完成 ORB-SLAM3 系统在 Windows 平台下的部署，并通过公开数据集验证其单目 SLAM 功能。因此，整个软件过程以“实验验证”为中心展开，特别强化了测试与结果分析相关环节。在开发过程中，更关注系统运行效果与轨迹输出的合理性，而非复杂功能扩展，从而保证软件过程与实验目标高度一致。

5.3 成本与效率平衡

在软件过程设计中，需要在规范性与开发效率之间进行权衡。本项目通过简化文档编写、弱化部分管理流程以及减少非必要的过程环节，有效降低了时间与人力成本。同时，将更多精力集中在环境配置、代码调试与实验验证上，提高了整体开发与问题解决效率，使项目能够在有限时间内顺利完成。

5.4 存在不足

- ◆ 文档记录较为简单，缺乏系统化整理，不利于后续复现与知识沉淀
- ◆ 测试过程主要依赖实验运行结果，缺乏结构化测试用例设计与系统性测试流程
- ◆ 未建立正式的软件质量评估机制，无法从规范角度对系统质量进行量化分析

这些不足在一定程度上限制了软件过程的规范性与工程化程度,也为后续改进与进一步完善提供了方向。

六、小结

通过本次作业,可以得出:

- ◆ 软件过程具有可裁剪性
- ◆ 标准是指导框架而非固定流程
- ◆ 实际项目需在规范与效率之间取得平衡